4월 23일 [판매원 데이터베이스]3장 연습문제 21번 기준

판매원 테이블
Salesperson(name, age, salary)
고객 테이블
Customer(name, city, industrytype)
주문 테이블
Order(number, custname, salesperson, amount)

```
-- 컬럼 설명
name          -- 판매원/고객 이름
age           -- 판매원 나이
salary        -- 판매원 급여
city          -- 고객 거주 도시
industrytype -- 고객 직업
number        -- 주문번호
custname      -- 주문 고객 이름
salesperson  -- 담당 판매원 이름
amount        -- 주문 금액
```

문제 1.
모든 판매원의 이름, 나이, 급여를 보여주는 뷰 v_salesperson_info를 작성하시오.

```
CREATE OR REPLACE VIEW v_salesperson_info AS
SELECT name AS 이름,
       age AS 나이,
       salary AS 급여
FROM Salesperson;
```

문제 2.
급여가 10,000원 이상인 판매원의 이름과 급여를 보여주는
읽기 전용 뷰 v_high_salary_sp를 작성하시오.

```
CREATE OR REPLACE VIEW v_high_salary_sp AS
SELECT name AS 이름,
       salary AS 급여
FROM Salesperson
WHERE salary >= 10000
WITH READ ONLY;
```

문제 3.
나이가 30세 미만인 판매원의 이름, 나이, 급여를 보여주는
뷰 v_young_salesperson을 작성하시오.

```
CREATE OR REPLACE VIEW v_young_salesperson AS
SELECT name AS 이름,
        age AS 나이,
        salary AS 급여
FROM Salesperson
WHERE age < 30;
```

문제 4.
'LA'에 거주하는 고객의 이름, 도시, 직업을 보여주는
읽기 전용 뷰 v_la_customer를 작성하시오.

```
CREATE OR REPLACE VIEW v_la_customer AS
SELECT name AS 이름,
        city AS 도시,
        industrytype AS 직업
FROM Customer
WHERE city = 'LA'
WITH READ ONLY;
```

문제 5.
직업이 '개발자'인 고객의 이름, 도시, 직업을 보여주는
뷰 v_developer_customer를 작성하시오.
단, 조건을 벗어나는 데이터는 입력할 수 없도록 설정하시오.

```
CREATE OR REPLACE VIEW v_developer_customer AS
SELECT name AS 이름,
        city AS 도시,
        industrytype AS 직업
FROM Customer
WHERE industrytype = '개발자'
WITH CHECK OPTION;
```

문제 6.
주문 금액이 15,000원 이상인 주문의
주문번호, 고객이름, 담당판매원, 주문금액을 보여주는
읽기 전용 뷰 v_high_amount_order를 작성하시오.

```
CREATE OR REPLACE VIEW v_high_amount_order AS
SELECT number AS 주문번호,
        custname AS 고객이름,
        salesperson AS 담당판매원,
        amount AS 주문금액
FROM Order
WHERE amount >= 15000
WITH READ ONLY;
```

문제 7.
급여가 8,000원 이상 12,000원 이하인 판매원의
이름, 나이, 급여를 보여주는 뷰 v_mid_salary_sp를 작성하시오.
단, 조건을 벗어나는 데이터는 입력할 수 없도록 설정하시오.

```
CREATE OR REPLACE VIEW v_mid_salary_sp AS
SELECT name AS 이름,
        age AS 나이,
        salary AS 급여
FROM Salesperson
WHERE salary BETWEEN 8000 AND 12000
WITH CHECK OPTION;
```

문제 8.
담당 판매원이 'Tom'인 주문의
주문번호, 고객이름, 담당판매원, 주문금액을 보여주는
뷰 v_tom_order를 작성하시오.

```
CREATE OR REPLACE VIEW v_tom_order AS
SELECT number AS 주문번호,
        custname AS 고객이름,
        salesperson AS 담당판매원,
        amount AS 주문금액
FROM Order
WHERE salesperson = 'Tom';
```

문제 9.
이름이 'S'로 시작하는 판매원의 이름, 나이, 급여를 보여주는
읽기 전용 뷰 v_s_salesperson을 작성하시오.

```
CREATE OR REPLACE VIEW v_s_salesperson AS
SELECT name AS 이름,
```

```
        age AS 나이,
        salary AS 급여
FROM Salesperson
WHERE name LIKE 'S%'
WITH READ ONLY;
```

문제 10.
주문 금액이 5,000원 이상 10,000원 이하인 주문의
주문번호, 고객이름, 담당판매원, 주문금액을 보여주는 뷰 v_mid_amount_order를 작성하시오. 단, 조건을 벗어나는 데이터는 입력할 수 없도록 설정하시오.

```
CREATE OR REPLACE VIEW v_mid_amount_order AS
SELECT number AS 주문번호,
        custname AS 고객이름,
        salesperson AS 담당판매원,
        amount AS 주문금액
FROM Order
WHERE amount BETWEEN 5000 AND 10000
WITH CHECK OPTION;
```

핵심 포인트 정리

문제	주요 개념
1	기본 단순 뷰 생성
2	WHERE 조건 + WITH READ ONLY
3	비교 연산자 (미만)
4	문자열 조건 + WITH READ ONLY
5	문자열 조건 + WITH CHECK OPTION
6	숫자 범위 조건 + WITH READ ONLY
7	BETWEEN 조건 + WITH CHECK OPTION
8	문자열 일치 조건
9	LIKE 연산자 + WITH READ ONLY
10	BETWEEN 조건 + WITH CHECK OPTION

(복합 뷰)
문제 1.
판매원별 총 주문금액과 주문 횟수를 보여주는 뷰 v_sp_order_summary를 작성하시오.
(출력 컬럼 : 판매원이름, 총주문금액, 주문횟수)

```
CREATE OR REPLACE VIEW v_sp_order_summary AS
SELECT s.name AS 판매원이름,
        SUM(o.amount) AS 총주문금액,
        COUNT(o.number) AS 주문횟수
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name;
```

문제 2.
고객별 총 주문금액과 주문 횟수를 보여주는 뷰 v_cust_order_summary를 작성하시오.
(출력 컬럼 : 고객이름, 도시, 총주문금액, 주문횟수)

```
CREATE OR REPLACE VIEW v_cust_order_summary AS
SELECT c.name AS 고객이름,
        c.city AS 도시,
        SUM(o.amount) AS 총주문금액,
        COUNT(o.number) AS 주문횟수
FROM Customer c
JOIN "Order" o ON c.name = o.custname
GROUP BY c.name, c.city;
```

문제 3.
판매원의 평균 급여보다 높은 급여를 받는 판매원의
이름, 나이, 급여를 보여주는 뷰 v_above_avg_salary를 작성하시오.

```
CREATE OR REPLACE VIEW v_above_avg_salary AS
SELECT name AS 이름,
        age AS 나이,
        salary AS 급여
FROM Salesperson
WHERE salary > (
    SELECT AVG(salary)
    FROM Salesperson
);
```

문제 4.

한 번도 주문을 받지 못한 판매원의
이름, 나이, 급여를 보여주는 뷰 v_no_order_sp를 작성하시오.

```
CREATE OR REPLACE VIEW v_no_order_sp AS
SELECT s.name AS 이름,
        s.age AS 나이,
        s.salary AS 급여
FROM Salesperson s
LEFT JOIN "Order" o ON s.name = o.salesperson
WHERE o.number IS NULL;
```

문제 5.
'LA'에 거주하는 고객으로부터 주문을 받은 판매원의
이름, 나이, 급여를 보여주는 뷰 v_la_order_sp를 작성하시오.

```
CREATE OR REPLACE VIEW v_la_order_sp AS
SELECT DISTINCT s.name AS 이름,
                s.age AS 나이,
                s.salary AS 급여
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
JOIN Customer c ON o.custname = c.name
WHERE c.city = 'LA';
```

문제 6.
판매원별 평균 주문금액을 계산하여
전체 평균 주문금액보다 높은 판매원의 이름과 평균주문금액을 보여주는
뷰 v_above_avg_order_sp를 작성하시오.

```
CREATE OR REPLACE VIEW v_above_avg_order_sp AS
SELECT s.name AS 판매원이름,
        ROUND(AVG(o.amount), 2) AS 평균주문금액
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name
HAVING AVG(o.amount) > (
    SELECT AVG(amount)
    FROM "Order"
);
```

문제 7.
한 번도 주문하지 않은 고객의

이름, 도시, 직업을 보여주는 뷰 v_no_order_cust를 작성하시오.

```
CREATE OR REPLACE VIEW v_no_order_cust AS
SELECT c.name AS 이름,
        c.city AS 도시,
        c.industrytype AS 직업
FROM Customer c
LEFT JOIN "Order" o ON c.name = o.custname
WHERE o.number IS NULL;
```

문제 8.

2건 이상 주문을 받은 판매원의

이름과 주문 횟수, 총 주문금액을 보여주는 뷰 v_frequent_sp를 작성하시오.

```
CREATE OR REPLACE VIEW v_frequent_sp AS
SELECT s.name AS 이름,
        COUNT(o.number) AS 주문횟수,
        SUM(o.amount) AS 총주문금액
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name
HAVING COUNT(o.number) >= 2;
```

문제 9.

고객의 도시별 총 주문금액과 주문 건수를 보여주는

뷰 v_city_order_stats를 작성하시오.

(출력 컬럼 : 도시, 총주문금액, 주문건수)

```
CREATE OR REPLACE VIEW v_city_order_stats AS
SELECT c.city AS 도시,
        SUM(o.amount) AS 총주문금액,
        COUNT(o.number) AS 주문건수
FROM Customer c
JOIN "Order" o ON c.name = o.custname
GROUP BY c.city;
```

문제 10.

판매원별 최고 주문금액, 최저 주문금액, 평균 주문금액과

자신의 급여 대비 총 주문금액 비율을 보여주는

뷰 v_sp_order_stats를 작성하시오.

(출력 컬럼 : 판매원이름, 급여, 최고주문금액, 최저주문금액, 평균주문금액, 급여대비주문비율)

```
CREATE OR REPLACE VIEW v_sp_order_stats AS
SELECT s.name AS 판매원이름,
        s.salary AS 급여,
        MAX(o.amount) AS 최고주문금액,
        MIN(o.amount) AS 최저주문금액,
        ROUND(AVG(o.amount), 2) AS 평균주문금액,
        ROUND(SUM(o.amount) / s.salary * 100, 2) AS 급여대비주문비율
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name, s.salary;
```

문제 11.
직업별 총 주문금액과 평균 주문금액을 보여주는
뷰 v_industry_order_stats를 작성하시오.
(출력 컬럼 : 직업, 총주문금액, 평균주문금액)

```
CREATE OR REPLACE VIEW v_industry_order_stats AS
SELECT c.industrytype AS 직업,
        SUM(o.amount) AS 총주문금액,
        ROUND(AVG(o.amount), 2) AS 평균주문금액
FROM Customer c
JOIN "Order" o ON c.name = o.custname
GROUP BY c.industrytype;
```

문제 12.
판매원 중 자신의 급여보다 총 주문금액이 더 높은 판매원의
이름, 급여, 총주문금액을 보여주는 뷰 v_sp_order_over_salary를 작성하시오.

```
CREATE OR REPLACE VIEW v_sp_order_over_salary AS
SELECT s.name AS 판매원이름,
        s.salary AS 급여,
        SUM(o.amount) AS 총주문금액
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name, s.salary
HAVING SUM(o.amount) > s.salary;
```

문제 13.
각 판매원이 주문을 받은 고객의 수(서로 다른 고객만)와

총 주문금액을 보여주는 뷰 v_sp_cust_count를 작성하시오.
(출력 컬럼 : 판매원이름, 담당고객수, 총주문금액)

```
CREATE OR REPLACE VIEW v_sp_cust_count AS
SELECT s.name AS 판매원이름,
        COUNT(DISTINCT o.custname) AS 담당고객수,
        SUM(o.amount) AS 총주문금액
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name;
```

문제 14.
주문 금액이 가장 높은 주문을 한 고객의
이름, 도시, 직업, 주문금액을 보여주는 뷰 v_max_order_cust를 작성하시오.

```
CREATE OR REPLACE VIEW v_max_order_cust AS
SELECT c.name AS 고객이름,
        c.city AS 도시,
        c.industrytype AS 직업,
        o.amount AS 주문금액
FROM Customer c
JOIN "Order" o ON c.name = o.custname
WHERE o.amount = (
    SELECT MAX(amount)
    FROM "Order"
);
```

문제 15.
판매원별로 주문금액의 합계를 구하고,
전체 주문금액 대비 각 판매원의 주문 비중을 보여주는
뷰 v_sp_order_ratio를 작성하시오.
(출력 컬럼 : 판매원이름, 총주문금액, 전체주문금액, 주문비중)

```
CREATE OR REPLACE VIEW v_sp_order_ratio AS
SELECT s.name AS 판매원이름,
        SUM(o.amount) AS 총주문금액,
        (SELECT SUM(amount) FROM "Order") AS 전체주문금액,
        ROUND(SUM(o.amount) / (SELECT SUM(amount) FROM "Order") * 100, 2) AS
주문비중
FROM Salesperson s
JOIN "Order" o ON s.name = o.salesperson
GROUP BY s.name;
```

핵심 포인트 정리

문제	주요 개념
1	JOIN + GROUP BY + SUM/COUNT
2	JOIN + GROUP BY + 다중 컬럼
3	스칼라 서브쿼리 + AVG 비교
4	LEFT JOIN + IS NULL (미수주)
5	3테이블 JOIN + DISTINCT
6	HAVING + 스칼라 서브쿼리
7	LEFT JOIN + IS NULL (미주문)
8	HAVING COUNT 조건
9	JOIN + GROUP BY 도시별 집계
10	MAX/MIN/AVG + 비율 계산
11	JOIN + GROUP BY 직업별 집계
12	HAVING으로 급여 대비 주문금액 비교
13	COUNT(DISTINCT) 중복 제거 고객 수
14	서브쿼리 + MAX로 최고 주문금액 비교
15	스칼라 서브쿼리로 전체합계 비율 계산

[극장 데이터베이스] 3장 연습문제 20번 기준

■ 테이블 구조와 데이터

극장

극장번호	극장이름	위치
1	롯데	잠실
2	메가	강남
3	대한	잠실

상영관

극장번호	상영관번호	영화제목	가격	좌석수
1	1	어려운 영화	15000	48
1	1	멋진 영화	7500	120
3	2	재밌는 영화	8000	110

예약

극장번호	상영관번호	고객번호	좌석번호	날짜
3	2	3	15	2025/09/01
3	1	4	16	2025/09/01
1	1	9	48	2025/09/01

고객

고객번호	이름	주소
3	홍길동	강남
4	김철수	잠실
9	박영희	강남

문제 1.
모든 극장의 극장번호, 극장이름, 위치를 보여주는 뷰 v_theater_info를 작성하시오.

```
CREATE OR REPLACE VIEW v_theater_info AS
SELECT 극장번호,
       극장이름,
       위치
FROM 극장;
```

문제 2.
위치가 '강남'인 극장의 극장번호, 극장이름을 보여주는
읽기 전용 뷰 v_gangnam_theater를 작성하시오.

```
CREATE OR REPLACE VIEW v_gangnam_theater AS
SELECT 극장번호,
       극장이름
FROM 극장
WHERE 위치 = '강남'
WITH READ ONLY;
```

문제 3.
가격이 10,000원 이상인 상영관의 극장번호, 상영관번호, 영화제목, 가격을 보여주는
읽기 전용 뷰 v_high_price_screen을 작성하시오.

```
CREATE OR REPLACE VIEW v_high_price_screen AS
SELECT 극장번호,
       상영관번호,
```

```
        영화제목,
        가격
FROM 상영관
WHERE 가격 >= 10000
WITH READ ONLY;
```

문제 4.
좌석수가 100석 이상인 상영관의 극장번호, 상영관번호, 영화제목, 좌석수를 보여주는 뷰 v_large_screen을 작성하시오.

```
CREATE OR REPLACE VIEW v_large_screen AS
SELECT 극장번호,
        상영관번호,
        영화제목,
        좌석수
FROM 상영관
WHERE 좌석수 >= 100;
```

문제 5.
주소가 '강남'인 고객의 고객번호, 이름, 주소를 보여주는 읽기 전용 뷰 v_gangnam_customer 를 작성하시오.

```
CREATE OR REPLACE VIEW v_gangnam_customer AS
SELECT 고객번호,
        이름,
        주소
FROM 고객
WHERE 주소 = '강남'
WITH READ ONLY;
```

문제 6.
2025년 9월 1일에 예약된 내역의 극장번호, 상영관번호, 고객번호, 좌석번호, 날짜를 보여주는 뷰 v_reservation_20250901을 작성하시오.

```
CREATE OR REPLACE VIEW v_reservation_20250901 AS
SELECT 극장번호,
        상영관번호,
        고객번호,
        좌석번호,
        날짜
```

```
FROM 예약
WHERE 날짜 = DATE '2025-09-01';
```

문제 7.
가격이 7,500원 이상 10,000원 이하인 상영관의 극장번호, 상영관번호, 영화제목, 가격을 보여주는 뷰 v_mid_price_screen을 작성하시오.
단, 조건을 벗어나는 데이터는 입력할 수 없도록 설정하시오.

```
CREATE OR REPLACE VIEW v_mid_price_screen AS
SELECT 극장번호,
       상영관번호,
       영화제목,
       가격
FROM 상영관
WHERE 가격 BETWEEN 7500 AND 10000
WITH CHECK OPTION;
```

문제 8.
좌석번호가 20번 이하인 예약 내역의 극장번호, 상영관번호, 고객번호, 좌석번호를 보여주는 뷰 v_front_seat_reservation을 작성하시오.

```
CREATE OR REPLACE VIEW v_front_seat_reservation AS
SELECT 극장번호,
       상영관번호,
       고객번호,
       좌석번호
FROM 예약
WHERE 좌석번호 <= 20;
```

문제 9.
영화제목에 '영화'가 포함된 상영관의 극장번호, 상영관번호, 영화제목, 가격, 좌석수를 보여주는 읽기 전용 뷰 v_movie_screen을 작성하시오.

```
CREATE OR REPLACE VIEW v_movie_screen AS
SELECT 극장번호,
       상영관번호,
       영화제목,
       가격,
       좌석수
FROM 상영관
```

```
WHERE 영화제목 LIKE '%영화%'
WITH READ ONLY;
```

문제 10.
주소가 '잠실'인 고객의 고객번호, 이름, 주소를 보여주는 뷰 v_jamsil_customer를 작성하시오. 단, 조건을 벗어나는 데이터는 입력할 수 없도록 설정하시오.

```
CREATE OR REPLACE VIEW v_jamsil_customer AS
SELECT 고객번호,
       이름,
       주소
FROM 고객
WHERE 주소 = '잠실'
WITH CHECK OPTION;
```

핵심 포인트 정리

문제	주요 개념
1	기본 단순 뷰 생성
2	문자열 조건 + WITH READ ONLY
3	숫자 범위 조건 + WITH READ ONLY
4	숫자 비교 조건
5	문자열 일치 조건 + WITH READ ONLY
6	DATE 날짜 조건
7	BETWEEN 조건 + WITH CHECK OPTION
8	숫자 비교 조건 (이하)
9	LIKE 연산자 + WITH READ ONLY
10	문자열 조건 + WITH CHECK OPTION

(극장 데이터베이스 복합 뷰 문제)

극장(극장번호, 극장이름, 위치)
상영관(극장번호, 상영관번호, 영화제목, 가격, 좌석수)
고객(고객번호, 이름, 주소)
예약(극장번호, 상영관번호, 고객번호, 좌석번호, 날짜)

문제 1.
극장별 상영관 수와 평균 가격을 보여주는 뷰 v_theater_screen_stats를 작성하시오.
(출력 컬럼 : 극장이름, 위치, 상영관수, 평균가격)

```
CREATE OR REPLACE VIEW v_theater_screen_stats AS
SELECT t.극장이름,
        t.위치,
        COUNT(s.상영관번호) AS 상영관수,
        ROUND(AVG(s.가격), 2) AS 평균가격
FROM 극장 t
JOIN 상영관 s ON t.극장번호 = s.극장번호
GROUP BY t.극장이름, t.위치;
```

문제 2.
각 극장의 상영관 중 가격이 가장 비싼 영화제목과 가격을 보여주는
뷰 v_theater_max_price를 작성하시오.
(출력 컬럼 : 극장이름, 영화제목, 가격)

```
CREATE OR REPLACE VIEW v_theater_max_price AS
SELECT t.극장이름,
        s.영화제목,
        s.가격
FROM 극장 t
JOIN 상영관 s ON t.극장번호 = s.극장번호
WHERE s.가격 = (
    SELECT MAX(s2.가격)
    FROM 상영관 s2
    WHERE s2.극장번호 = s.극장번호
);
```

문제 3.
고객별 총 예약 횟수를 보여주는 뷰 v_customer_reservation_count를 작성하시오.
(출력 컬럼 : 고객이름, 주소, 총예약횟수)

```
CREATE OR REPLACE VIEW v_customer_reservation_count AS
SELECT c.이름 AS 고객이름,
       c.주소,
       COUNT(r.고객번호) AS 총예약횟수
FROM 고객 c
JOIN 예약 r ON c.고객번호 = r.고객번호
GROUP BY c.이름, c.주소;
```

문제 4.
한 번도 예약하지 않은 고객의 고객번호, 이름, 주소를 보여주는
뷰 v_no_reservation_customer를 작성하시오.

```
CREATE OR REPLACE VIEW v_no_reservation_customer AS
SELECT c.고객번호,
       c.이름,
       c.주소
FROM 고객 c
LEFT JOIN 예약 r ON c.고객번호 = r.고객번호
WHERE r.고객번호 IS NULL;
```

문제 5.
예약이 한 번도 없는 상영관의 극장번호, 상영관번호, 영화제목, 가격을 보여주는 뷰 v_no_reservation_screen을 작성하시오.

```
CREATE OR REPLACE VIEW v_no_reservation_screen AS
SELECT s.극장번호,
       s.상영관번호,
       s.영화제목,
       s.가격
FROM 상영관 s
LEFT JOIN 예약 r ON s.극장번호 = r.극장번호
                AND s.상영관번호 = r.상영관번호
WHERE r.극장번호 IS NULL;
```

문제 6.
극장별 총 예약 건수와 총 예약 좌석수를 보여주는 뷰 v_theater_reservation_stats를 작성하시오. (출력 컬럼 : 극장이름, 총예약건수, 총예약좌석수)

```
CREATE OR REPLACE VIEW v_theater_reservation_stats AS
SELECT t.극장이름,
       COUNT(r.좌석번호) AS 총예약건수,
       SUM(r.좌석번호) AS 총예약좌석수
FROM 극장 t
JOIN 예약 r ON t.극장번호 = r.극장번호
GROUP BY t.극장이름;
```

문제 7.
'강남'에 사는 고객이 예약한 내역의 고객이름, 극장번호, 상영관번호, 좌석번호, 날짜를 보여주는 뷰 v_gangnam_customer_reservation을 작성하시오.

```
CREATE OR REPLACE VIEW v_gangnam_customer_reservation AS
SELECT c.이름 AS 고객이름,
       r.극장번호,
       r.상영관번호,
       r.좌석번호,
       r.날짜
FROM 고객 c
JOIN 예약 r ON c.고객번호 = r.고객번호
WHERE c.주소 = '강남';
```

문제 8.
상영관별 예약 건수와 전체 좌석수 대비 예약된 좌석의 비율을 보여주는
뷰 v_screen_reservation_rate를 작성하시오.
(출력 컬럼 : 극장번호, 상영관번호, 영화제목, 좌석수, 예약건수, 예약률)
단, 예약률은 소수점 2자리까지 표시한다.

```
CREATE OR REPLACE VIEW v_screen_reservation_rate AS
SELECT s.극장번호,
       s.상영관번호,
       s.영화제목,
       s.좌석수,
       COUNT(r.좌석번호) AS 예약건수,
```

```
        ROUND(COUNT(r.좌석번호) / s.좌석수 * 100, 2) AS 예약률
FROM 상영관 s
LEFT JOIN 예약 r ON s.극장번호 = r.극장번호
                AND s.상영관번호 = r.상영관번호
GROUP BY s.극장번호, s.상영관번호, s.영화제목, s.좌석수;
```

단계별 풀이

STEP 1. 어떤 테이블이 필요한가?

필요한 정보	테이블
극장번호, 상영관번호, 영화제목, 좌석수	상영관
예약 건수	예약

→ **상영관** 테이블과 **예약** 테이블을 JOIN

STEP 2. 왜 LEFT JOIN인가?

```sql
FROM 상영관 s
LEFT JOIN 예약 r ON s.극장번호 = r.극장번호
                AND s.상영관번호 = r.상영관번호
```

예약이 **한 건도 없는 상영관도** 결과에 포함시켜야 하기 때문입니다.

```
-- 일반 JOIN 사용 시
상영관     예약
  A    →  있음  ✅ 출력됨
  B    →  없음  ❌ 출력 안됨  (문제!)

-- LEFT JOIN 사용 시
상영관     예약
  A    →  있음  ✅ 출력됨
  B    →  없음  ✅ 출력됨 (예약건수 = 0으로 표시)
```

STEP 3. 왜 복합키로 JOIN하는가?

```sql
ON s.극장번호 = r.극장번호
AND s.상영관번호 = r.상영관번호
```

상영관 테이블의 기본키가 **극장번호 + 상영관번호** 의 복합키이기 때문입니다.

예를 들어 극장번호=1, 상영관번호=1과 극장번호=3, 상영관번호=1은 **서로 다른 상영관**이므로 두 컬럼을 모두 JOIN 조건으로 사용해야 합니다.

```
극장번호=1, 상영관번호=1 → 어려운 영화
극장번호=3, 상영관번호=1 → 멋진 영화    ← 다른 상영관!
```

STEP 4. COUNT와 GROUP BY

```sql
COUNT(r.좌석번호) AS 예약건수
```

- `COUNT(r.좌석번호)` → 예약 테이블의 좌석번호가 **NULL이 아닌 행의 수**를 셈
- 예약이 없는 상영관은 LEFT JOIN으로 인해 좌석번호가 NULL이므로 → `COUNT = 0` 으로 자동 처리됨

```sql
GROUP BY s.극장번호, s.상영관번호, s.영화제목, s.좌석수
```

상영관별로 집계하기 위해 **상영관을 식별하는 모든 컬럼**을 GROUP BY에 포함시킵니다.

STEP 5. 예약률 계산

```sql
ROUND(COUNT(r.좌석번호) / s.좌석수 * 100, 2) AS 예약률
```

항목	설명
`COUNT(r.좌석번호)`	예약된 건수
`s.좌석수`	전체 좌석수
`/ s.좌석수 * 100`	백분율(%) 계산
`ROUND(..., 2)`	소수점 2자리 반올림

STEP 6. 실제 데이터로 결과 확인

테이블 데이터

극장번호	상영관번호	영화제목	좌석수
1	1	어려운 영화	48
1	1	멋진 영화	120
3	2	재밌는 영화	110

예약 데이터

극장번호	상영관번호	고객번호	좌석번호
3	2	3	15
3	1	4	16
1	1	9	48

뷰 실행 결과

극장번호	상영관번호	영화제목	좌석수	예약건수	예약률
1	1	어려운 영화	48	1	2.08
1	1	멋진 영화	120	0	0.00
3	2	재밌는 영화	110	1	0.91

문제 9.
2건 이상 예약한 고객의 이름, 주소, 예약횟수를 보여주는 뷰 v_frequent_customer를 작성하시오.

```
CREATE OR REPLACE VIEW v_frequent_customer AS
SELECT c.이름,
        c.주소,
        COUNT(r.고객번호) AS 예약횟수
FROM 고객 c
JOIN 예약 r ON c.고객번호 = r.고객번호
GROUP BY c.이름, c.주소
HAVING COUNT(r.고객번호) >= 2;
```

풀이과정 중 중요 사항

STEP 2. JOIN 조건

```sql
FROM 고객 c
JOIN 예약 r ON c.고객번호 = r.고객번호
```

- 고객 테이블의 `고객번호`와 예약 테이블의 `고객번호`를 연결
- 예약한 고객만 필요하므로 **일반 JOIN** 사용 (예약 없는 고객은 출력 불필요)

STEP 3. COUNT와 GROUP BY

```sql
COUNT(r.고객번호) AS 예약횟수
GROUP BY c.이름, c.주소
```

고객별로 예약 횟수를 집계하기 위해 **고객을 식별하는 컬럼**을 GROUP BY에 포함시킵니다.

```
고객번호=3 홍길동  → 예약 2건
고객번호=4 김철수  → 예약 1건
고객번호=9 박영희  → 예약 1건
```

STEP 4. WHERE vs HAVING

2건 이상 조건을 적용할 때 **WHERE가 아닌 HAVING**을 사용합니다.

```sql
-- ❌ WHERE 사용 불가 (집계함수는 WHERE에 사용 불가)
WHERE COUNT(r.고객번호) >= 2

-- ✅ HAVING 사용 (집계함수 조건은 HAVING에 사용)
HAVING COUNT(r.고객번호) >= 2
```

구분	WHERE	HAVING
적용 시점	GROUP BY **이전** 필터링	GROUP BY **이후** 필터링
집계함수 사용	❌ 불가	✅ 가능
사용 목적	행 단위 조건	그룹 단위 조건

집계함수(COUNT) 조건은 HAVING에 작성한다.

문제 10.
영화제목별 총 예약 건수와 총 예약 금액을 보여주는 뷰 v_movie_reservation_stats를 작성하시오. (출력 컬럼 : 영화제목, 총예약건수, 총예약금액)

```
CREATE OR REPLACE VIEW v_movie_reservation_stats AS
SELECT s.영화제목,
       COUNT(r.좌석번호) AS 총예약건수,
       SUM(s.가격) AS 총예약금액
FROM 상영관 s
JOIN 예약 r ON s.극장번호 = r.극장번호
           AND s.상영관번호 = r.상영관번호
GROUP BY s.영화제목;
```

문제 11.
극장 위치별 평균 영화 가격과 총 상영관 수를 보여주는 뷰 v_location_screen_stats를 작성하시오. (출력 컬럼 : 위치, 평균가격, 총상영관수)

```
CREATE OR REPLACE VIEW v_location_screen_stats AS
SELECT t.위치,
       ROUND(AVG(s.가격), 2) AS 평균가격,
       COUNT(s.상영관번호) AS 총상영관수
FROM 극장 t
JOIN 상영관 s ON t.극장번호 = s.극장번호
GROUP BY t.위치;
```

문제 12.
각 고객이 가장 최근에 예약한 날짜와 예약한 극장번호, 상영관번호를 보여주는 뷰 v_customer_last_reservation을 작성하시오.
(출력 컬럼 : 고객이름, 극장번호, 상영관번호, 최근예약날짜)

```
CREATE OR REPLACE VIEW v_customer_last_reservation AS
SELECT c.이름 AS 고객이름,
       r.극장번호,
       r.상영관번호,
       r.날짜 AS 최근예약날짜
FROM 고객 c
JOIN 예약 r ON c.고객번호 = r.고객번호
```

```
WHERE r.날짜 = (
    SELECT MAX(r2.날짜)
    FROM 예약 r2
    WHERE r2.고객번호 = r.고객번호
);
```

단계별 풀이

STEP 1. 어떤 테이블이 필요한가?

필요한 정보	테이블
고객이름	고객
극장번호, 상영관번호, 날짜	예약

→ **고객** 테이블과 **예약** 테이블을 JOIN

STEP 3. 핵심 - 서브쿼리로 최근 날짜 찾기

```sql
WHERE r.날짜 = (
    SELECT MAX(r2.날짜)
    FROM 예약 r2
    WHERE r2.고객번호 = r.고객번호
)
```

서브쿼리 동작 방식

```
-- 고객번호 = 3 (홍길동)일 때 서브쿼리 실행
SELECT MAX(r2.날짜)
FROM 예약 r2
WHERE r2.고객번호 = 3
→ 결과 : 2025/09/01

-- 고객번호 = 4 (김철수)일 때 서브쿼리 실행
SELECT MAX(r2.날짜)
FROM 예약 r2
WHERE r2.고객번호 = 4
→ 결과 : 2025/09/01

-- 고객번호 = 9 (박영희)일 때 서브쿼리 실행
SELECT MAX(r2.날짜)
FROM 예약 r2
WHERE r2.고객번호 = 9
→ 결과 : 2025/09/01
```

STEP 4. 상관 서브쿼리란?

```sql
-- 일반 서브쿼리 : 외부 쿼리와 무관하게 한 번만 실행
WHERE 날짜 = (SELECT MAX(날짜) FROM 예약)
-- → 전체에서 가장 최근 날짜 1개만 반환 (고객 구분 없음) ❌

-- 상관 서브쿼리 : 외부 쿼리의 각 행마다 반복 실행
WHERE r.날짜 = (SELECT MAX(r2.날짜) FROM 예약 r2
                WHERE r2.고객번호 = r.고객번호)
-- → 각 고객별로 최근 날짜를 개별 계산 ✅
```

구분	일반 서브쿼리	상관 서브쿼리
실행 횟수	1번만 실행	외부 행마다 반복 실행
외부 참조	❌ 없음	✅ 외부 쿼리 값 참조
사용 목적	단순 비교	행마다 다른 조건 적용

STEP 5. 쿼리 실행 순서

```
① FROM 고객 c
        ↓
② JOIN 예약 r ON c.고객번호 = r.고객번호
        ↓
③ 각 행마다 서브쿼리 실행
   → 해당 고객의 MAX(날짜) 계산
        ↓
④ WHERE r.날짜 = MAX(날짜) 조건 적용
   → 최근 날짜와 일치하는 행만 필터링
        ↓
⑤ SELECT 고객이름, 극장번호, 상영관번호, 날짜 출력
```

7

문제 13.

전체 평균 가격보다 비싼 상영관의 극장 이름, 상영관번호, 영화제목, 가격을 보여주는 뷰 v_above_avg_price_screen을 작성하시오.

```
CREATE OR REPLACE VIEW v_above_avg_price_screen AS
SELECT t.극장이름,
       s.상영관번호,
       s.영화제목,
       s.가격
FROM 상영관 s
JOIN 극장 t ON s.극장번호 = t.극장번호
WHERE s.가격 > (
    SELECT AVG(가격)
    FROM 상영관
);
```

문제 14.
극장별로 예약 건수가 가장 많은 상영관의 극장이름, 상영관번호, 영화제목, 예약건수를 보여주는 뷰 v_most_reserved_screen을 작성하시오.

```
CREATE OR REPLACE VIEW v_most_reserved_screen AS
SELECT t.극장이름,
       s.상영관번호,
       s.영화제목,
       COUNT(r.좌석번호) AS 예약건수
FROM 극장 t
JOIN 상영관 s ON t.극장번호 = s.극장번호
JOIN 예약 r ON s.극장번호 = r.극장번호
           AND s.상영관번호 = r.상영관번호
GROUP BY t.극장이름, s.상영관번호, s.영화제목
HAVING COUNT(r.좌석번호) = (
    SELECT MAX(COUNT(r2.좌석번호))
    FROM 예약 r2
    WHERE r2.극장번호 = s.극장번호
    GROUP BY r2.상영관번호
);
```

문제 15.
고객별 총 예약금액을 계산하고 전체 예약금액 대비 각 고객의 예약 비중을 보여주는
뷰 v_customer_payment_ratio를 작성하시오.
(출력 컬럼 : 고객이름, 총예약금액, 전체예약금액, 예약비중)

```
CREATE OR REPLACE VIEW v_customer_payment_ratio AS
SELECT c.이름 AS 고객이름,
       SUM(s.가격) AS 총예약금액,
       (SELECT SUM(s2.가격)
        FROM 상영관 s2
        JOIN 예약 r2 ON s2.극장번호 = r2.극장번호
                    AND s2.상영관번호 = r2.상영관번호) AS 전체예약금액,
       ROUND(SUM(s.가격) /
           (SELECT SUM(s2.가격)
            FROM 상영관 s2
            JOIN 예약 r2 ON s2.극장번호 = r2.극장번호
                        AND s2.상영관번호 = r2.상영관번호) * 100, 2) AS 예약비중
FROM 고객 c
JOIN 예약 r ON c.고객번호 = r.고객번호
JOIN 상영관 s ON r.극장번호 = s.극장번호
            AND r.상영관번호 = s.상영관번호
GROUP BY c.이름;
```

***where과 having 의 차이점

: count, sum과 같은 집계함수 조건은 무조건 having

: =,>,<, like 등 일반조건은 무조건 where

*** group by를 쓰는 경우는?

집계함수를 쓸 때만 사용!

having을 쓰는 경우에만 group by를 사용.

.

핵심 포인트 정리

문제	주요 개념
1	JOIN + GROUP BY + COUNT/AVG
2	JOIN + 서브쿼리 MAX 비교
3	JOIN + GROUP BY + COUNT
4	LEFT JOIN + IS NULL (미예약)
5	LEFT JOIN + 복합키 IS NULL
6	JOIN + GROUP BY + COUNT/SUM
7	JOIN + WHERE 주소 조건
8	LEFT JOIN + 복합키 + 예약률 계산
9	JOIN + HAVING COUNT 조건
10	JOIN + 복합키 + GROUP BY
11	JOIN + GROUP BY 위치별 집계
12	JOIN + 서브쿼리 MAX 날짜 비교
13	JOIN + 스칼라 서브쿼리 AVG 비교
14	JOIN + HAVING MAX 서브쿼리
15	스칼라 서브쿼리로 전체합계 비율 계산